



TRABAJO FINAL DE Ingeniería en Sistemas de Información

Orquestación de agentes de lenguaje para el desarrollo de
software: una arquitectura basada en contratos y
descomposición adaptativa con verificación basada en evidencia

Alumno: Francisco Clarke (LU: 120547)

Directora: Dra. Ana Gabriela Maguitman

Codirector: Dr. Federico Martín Schmidt

Bahía Blanca, Argentina

2026

Resumen

Los agentes de programación basados en modelos de lenguaje pueden inspeccionar repositorios, modificar archivos y ejecutar pruebas. Esta capacidad resulta útil para cambios acotados, pero enfrenta una dificultad cuando el objetivo atraviesa varios módulos: si todo el trabajo se delega a un único agente, su contexto se vuelve difícil de administrar; si se divide en demasiadas partes, el costo de coordinación y de integración puede superar el beneficio de la división. En esta tesis se denomina *paradoja de la granularidad* a esa tensión entre tareas demasiado grandes y fragmentaciones innecesarias.

Se presenta **ManyHands**, una plataforma local que coordina agentes de código para transformar un objetivo expresado en lenguaje natural en un cambio de software integrado, verificado y entregado en Git. Su diseño combina cinco estrategias. Primero, representa el trabajo mediante un grafo jerárquico que distingue el objetivo, las fronteras de integración y las unidades ejecutables. Segundo, reemplaza la coordinación informal por contratos versionados que fijan alcance, interfaces, artefactos y criterios de aceptación. Tercero, ejecuta cada intento en un *worktree* aislado y bajo supervisión del sistema operativo. Cuarto, integra los resultados de abajo hacia arriba y los valida sobre commits físicos exactos mediante una Matriz de Evidencias. Quinto, conserva el ciclo de vida en un registro de eventos de solo anexión, lo que permite reconstruir y auditar un run aun después de una interrupción.

Como aporte complementario se desarrolla una política categórica de granularidad. En lugar de combinar métricas heterogéneas en un puntaje opaco, la política acepta una división únicamente si existe al menos una razón observable: la tarea no cabe en un intento, las partes pueden ejecutarse en paralelo o cada parte puede verificarse de forma independiente. Cuando ninguna razón se cumple, el plan se colapsa a una hoja cohesiva sin perder criterios de aceptación.

La arquitectura se evaluó mediante regresiones deterministas y mediante el run *Viaje en Familia*. En este último caso, el sistema planificó un grafo de tres niveles, ejecutó ramas independientes, realizó integraciones intermedias, recuperó fallas de validación sin reiniciar el trabajo ya adoptado y publicó un commit final reproducible. El resultado demuestra la viabilidad de una cadena de orquestación auditable y ofrece una base concreta para estudiar sistemas de desarrollo autónomo en los que el modelo propone soluciones, pero el software determinista conserva la autoridad sobre la ejecución y la evidencia.

Palabras clave: Orquestación de agentes, modelos de lenguaje, descomposición de tareas, diseño por contratos, grafos de tareas, Git worktrees, verificación basada en evidencia, Event Sourcing.

Índice general

Resumen	i
1. Introducción	1
1.1. Motivación	1
1.2. El problema: la paradoja de la granularidad	1
1.3. Propuesta de solución	2
1.4. Preguntas y objetivos de investigación	2
1.5. Alcance	3
1.6. Organización del documento	3
2. Conceptos preliminares	4
2.1. Agentes de código	4
2.2. Grafos de tareas	4
2.3. Diseño por contratos	5
2.4. Aislamiento mediante Git worktrees	5
2.5. Verificación basada en evidencia	5
2.6. Eventos como historia canónica	6
3. Estado del arte	7
3.1. Agentes individuales de programación	7
3.2. Sistemas multi-agente conversacionales	7
3.3. Grafos y paralelismo	7
3.4. Asignación de modelos y granularidad	8
3.5. Posicionamiento de ManyHands	8
4. Política de granularidad adaptativa	9
4.1. La tensión fundamental: cuándo y cómo dividir	9
4.2. Crítica a los modelos continuos de scoring heurístico	9
4.2.1. El análisis empírico sobre 83 cortes históricos	10
4.3. La Política de Granularidad Categórica 4.0	10
4.3.1. Límites empíricos de viabilidad de hoja (<i>Leaf Feasibility Bounds</i>)	11
4.3.2. Las tres razones categóricas de división	11
4.3.3. Regla de decisión determinista	12
4.4. Algoritmo de colapso y remapeo de planes	12
4.5. Frontera entre semántica y control determinista	12
4.6. Metodología del banco de regresión offline	13

5. Arquitectura del sistema	14
5.1. Vista integral: del objetivo a la entrega	14
5.2. Modelo de grafo híbrido y relaciones canónicas tipadas	15
5.2.1. Tipado canónico de relaciones	15
5.3. Grounding del repositorio: RepositoryModel y consultas presupuestadas	16
5.4. Motor de planificación progresiva presupuestada (PlanningEngine)	17
5.4.1. Prevención de bucles causales (<i>Causal Loop Prevention</i>)	17
5.4.2. Taxonomía discriminada de resultados de planificación	17
5.5. Sistema canónico de contratos: acotando los modos de falla	17
5.6. Verificador estático de invariantes (verifyPlan): prevención de patologías de grafo	19
6. Implementación	20
6.1. Organización del código y fronteras de paquetes	20
6.2. Núcleo durable: eventos, actor y efectos	20
6.3. Disponibilidad, priorización y conducción de olas	21
6.4. Intentos, artefactos y ejecución aislada	21
6.4.1. Pool de Git worktrees y reciclado determinista	21
6.4.2. Verificación estricta de alcance en tiempo de ejecución (<i>Scope Guard</i>) .	22
6.4.3. Supervisión de procesos, comunicación IPC y saneamiento de entorno .	22
6.5. Integración ascendente y reparación semántica de conflictos	22
6.6. Recuperación causal y decisiones durante el run	23
6.7. Matriz de Evidencias (Evidence Matrix)	23
6.8. Publicación y Entrega Transaccional (TransactionalDeliveryPublisher)	24
6.9. Persistencia append-only basada en eventos y Daemon local	24
6.10. Interfaz web de supervisión y accesibilidad	25
7. Evaluación experimental: Viaje en Familia	26
7.1. Diseño del caso	26
7.2. Configuración y fuente de evidencia	26
7.3. Topología producida	26
7.4. Ejecución, paralelismo y recuperación	27
7.5. Integraciones y candidato final	27
7.6. Reproducción y alcance del resultado	28
8. Discusión	29
8.1. Qué demuestra el sistema	29
8.2. Modelos para semántica, software para autoridad	29
8.3. Por qué la granularidad es categórica	29
8.4. Costo de control y recuperación	30
8.5. Alcance de la evidencia	30
8.6. Amenazas a la validez	30
9. Conclusiones	31
9.1. Respuesta a las preguntas de investigación	31
9.2. Aportes principales	31
9.3. Lecciones de ingeniería	32
9.4. Trabajo futuro	32

Capítulo 1

Introducción

1.1. Motivación

Los modelos de lenguaje dejaron de ser herramientas que sólo completan líneas de código. Los agentes actuales pueden recorrer un repositorio, ejecutar comandos, interpretar errores y modificar varios archivos en ciclos sucesivos de observación y acción [6, 20, 19]. Esta evolución amplió el tipo de problemas que pueden abordar, pero también volvió visible una diferencia central: resolver un parche local no es equivalente a coordinar un cambio que atraviesa dominio, persistencia, interfaces y pruebas.

Un agente único puede perder información relevante cuando debe sostener a la vez demasiados archivos, dependencias y decisiones. Dividir el trabajo entre varios agentes parece una solución natural, pero introduce un problema nuevo: cada frontera creada debe coordinarse y luego integrarse. Dos agentes pueden implementar firmas incompatibles, editar el mismo archivo o aprobar pruebas que no demuestran el comportamiento solicitado. Por lo tanto, el desafío no es simplemente utilizar más agentes, sino gobernar su colaboración como un proceso de ingeniería de software.

Esta tesis aborda ese desafío mediante **ManyHands**, un plano de control local que coordina agentes de código sin reemplazarlos. Los modelos conservan aquello en lo que son valiosos—interpretar objetivos y proponer implementaciones—, mientras que el sistema determina qué puede ejecutarse, qué debe esperar, qué rutas puede modificar cada intento y qué evidencia se exige antes de entregar un resultado.

1.2. El problema: la paradoja de la granularidad

La modularidad busca dividir un sistema alrededor de responsabilidades que puedan cambiar de forma relativamente independiente [15]. En un sistema multi-agente, esta idea adquiere una consecuencia operacional: la forma de dividir el objetivo determina cuántos agentes se ejecutan, cuánto contexto recibe cada uno y cuántas integraciones deben resolverse.

Existen dos extremos igualmente problemáticos:

- **Una tarea demasiado grande.** Un solo agente debe comprender muchas decisiones y archivos. Aumentan la duración del intento, la pérdida de contexto y las modificaciones fuera de alcance.
- **Una tarea excesivamente fragmentada.** Cada fragmento requiere contexto, contrato, proceso y validación propios. Crecen las costuras entre partes y, con ellas, el riesgo de incompatibilidades y el costo de integración [2].

Se denomina **paradoja de la granularidad** a la necesidad de encontrar una división suficientemente pequeña para ser ejecutable, pero suficientemente cohesiva para que la coordinación no domine el trabajo útil. No existe un tamaño universal: la decisión depende de las fronteras del repositorio, de los artefactos compartidos y de la posibilidad de verificar cada parte.

1.3. Propuesta de solución

ManyHands trata un objetivo de software como un *run*: una transformación completa desde un repositorio base hasta un commit integrado y entregado. El run se organiza mediante cinco estrategias complementarias:

1. **Grafo híbrido.** La jerarquía expresa quién integra a quién; las relaciones tipadas expresan dependencias de artefactos, acuerdos de interfaz y conflictos de escritura.
2. **Contratos versionados.** Cada unidad conoce su objetivo, alcance, artefactos, interfaces y obligaciones de validación antes de ejecutar.
3. **Granularidad explicable.** Una división se conserva sólo si aporta capacidad, paralelismo o aislamiento de pruebas.
4. **Aislamiento y recuperación.** Los intentos se ejecutan en worktrees separados, bajo supervisión de procesos y con estado durable.
5. **Evidencia antes que declaración.** La integración y la entrega dependen de pruebas y recibos asociados al commit exacto, no del relato del modelo.

Estas estrategias forman una cadena de custodia: cada etapa recibe una entrada identificada, produce un artefacto verificable y deja una evidencia durable para la etapa siguiente.

1.4. Preguntas y objetivos de investigación

El trabajo se organiza alrededor de dos preguntas:

- PI-1** ¿Es viable coordinar agentes de código mediante un grafo y contratos versionados, aislando su ejecución e integrando sus resultados de manera que cada entrega quede vinculada a evidencia reproducible sobre un commit exacto?
- PI-2** ¿Puede una política determinista y explicable decidir cuándo dividir una tarea o conservarla como una unidad cohesiva a partir de propiedades observables de capacidad, paralelismo y verificación?

La hipótesis principal sostiene que una arquitectura basada en contratos, relaciones tipadas, aislamiento físico e integración por evidencia permite coordinar agentes heterogéneos sin depender de correcciones imperativas del grafo. La hipótesis complementaria sostiene que una política categórica puede evitar la microfragmentación sin recurrir a puntajes difíciles de interpretar.

El **objetivo general** es diseñar, implementar y evaluar una plataforma local de orquestación multi-agente que transforme objetivos de software en resultados integrados, auditables y reproducibles. De él se derivan los siguientes objetivos específicos:

1. modelar el trabajo mediante grafos jerárquicos y relaciones tipadas;
2. formalizar contratos de objetivo, alcance, interfaz, artefacto y validación;
3. diseñar una política categórica de granularidad y un algoritmo de colapso que preserve criterios;
4. implementar ejecución aislada, supervisión de procesos y recuperación durable;
5. integrar resultados de forma ascendente y construir evidencia sobre el candidato exacto;
6. publicar entregas transaccionales y conservar un historial auditable;
7. evaluar el ciclo completo mediante regresiones y un caso experimental representativo.

1.5. Alcance

ManyHands se diseñó como una herramienta local para un desarrollador. Opera sobre repositorios Git y utiliza ejecutores externos de modelos mediante CLI. El trabajo prioriza corrección, contención y trazabilidad por encima de la velocidad máxima. Quedan fuera de alcance la colaboración multiusuario, el entrenamiento de modelos, la optimización de costos entre proveedores y el aislamiento mediante máquinas virtuales completas.

La evaluación estudia la viabilidad de la arquitectura y su capacidad para producir una descomposición útil. No pretende establecer una comparación estadística universal entre modelos ni generalizar rendimiento a todos los lenguajes y tamaños de repositorio.

1.6. Organización del documento

Los capítulos 2 y 3 introducen los conceptos y antecedentes. El capítulo 4 presenta la política de granularidad. Los capítulos 5 y 6 explican el diseño y su materialización. El capítulo 7 describe el experimento *Viaje en Familia*; el capítulo 8 analiza sus alcances y el capítulo 9 sintetiza los aportes y líneas futuras.

Capítulo 2

Conceptos preliminares

Este capítulo introduce los conceptos necesarios para comprender el diseño de ManyHands. El propósito no es agotar cada tema, sino establecer un vocabulario común para explicar cómo se coordinan agentes, código y evidencia.

2.1. Agentes de código

Un agente de código combina un modelo de lenguaje con herramientas que le permiten observar y modificar un entorno de desarrollo. Su operación puede entenderse como un ciclo de cuatro pasos [20, 18]:

1. observa archivos, estructura y resultados de comandos;
2. interpreta el objetivo y decide una acción;
3. edita código o ejecuta una herramienta;
4. usa el resultado para continuar o corregirse.

Este ciclo explica por qué un agente puede resolver errores de compilación sin intervención constante. Sin embargo, no establece por sí mismo qué archivos puede tocar, cómo se coordina con otros agentes ni qué prueba demuestra que su trabajo es correcto. ManyHands agrega ese plano de control alrededor de los ejecutores existentes.

2.2. Grafos de tareas

Un grafo dirigido acíclico o DAG es un conjunto de nodos unidos por relaciones dirigidas sin ciclos. La ausencia de ciclos permite ordenar las tareas: si $A \rightarrow B$, entonces A debe producir algo antes de que B pueda continuar. Los nodos que no dependen entre sí forman una *frontera* y pueden ejecutarse al mismo tiempo.

En software, una única flecha de dependencia resulta ambigua. Dos tareas pueden estar relacionadas porque una consume el archivo de la otra, porque comparten una interfaz o simplemente porque pretenden escribir el mismo recurso. Estas situaciones producen efectos distintos: espera, paralelismo coordinado o exclusión mutua. Por esa razón ManyHands utiliza relaciones tipadas, descritas en el capítulo 5.

2.3. Diseño por contratos

El Diseño por Contratos propone que la colaboración entre componentes se base en obligaciones explícitas, no en supuestos implícitos [13]. En un sistema con modelos generativos este principio es especialmente útil: el modelo puede interpretar lenguaje natural, pero la ejecución necesita límites que un programa pueda validar.

ManyHands aplica contratos en cuatro dimensiones:

- **alcance:** qué rutas puede modificar una unidad;
- **interfaz o costura:** qué firma comparten productor y consumidores;
- **artefacto:** qué resultado material debe producirse;
- **validación:** qué comportamiento debe comprobarse y con qué evidencia.

Un contrato no garantiza que el agente escriba código correcto, pero convierte una expectativa difusa en una condición observable. También permite detectar incompatibilidades antes de consumir tiempo de ejecución.

2.4. Aislamiento mediante Git worktrees

Ejecutar varios agentes sobre el mismo directorio produciría condiciones de carrera y dificultaría atribuir cada cambio. Git ofrece los *worktrees*: directorios de trabajo independientes que comparten la base de objetos del repositorio, pero poseen índice y HEAD propios [4].

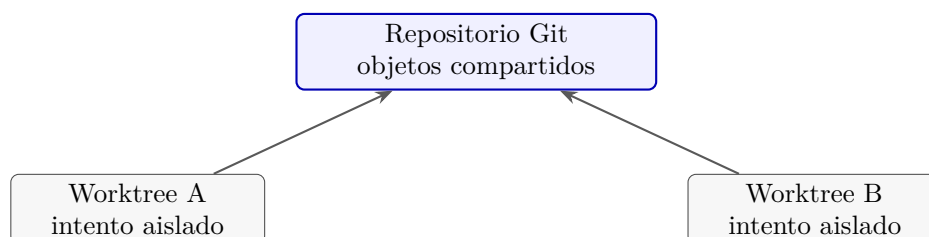


Figura 2.1: Dos intentos comparten objetos Git, pero no su directorio de trabajo.

El worktree evita interferencias entre cambios, pero no constituye por sí solo un sandbox de seguridad. ManyHands lo complementa con control de rutas, credenciales acotadas y supervisión de procesos. La separación conceptual es importante: el worktree protege la composición de código; el sandbox limita las capacidades del proceso.

2.5. Verificación basada en evidencia

La salida textual de un agente no es una prueba de corrección. La ingeniería de software utiliza compiladores, tests e integración continua para evaluar el artefacto observable [10, 1]. ManyHands lleva este principio a una cadena de evidencia asociada a identidades Git:

1. cada resultado se vincula al commit candidato exacto;
2. cada criterio se asocia con una obligación de validación;
3. se conserva el comando, su salida y su huella;

4. cuando corresponde, un control negativo comprueba que la prueba detecta realmente la ausencia del cambio.

La Matriz de Evidencias resume estas relaciones. Su función no es sustituir el juicio de producto, sino impedir que un estado verde pierda trazabilidad sobre qué se probó y sobre qué versión.

2.6. Eventos como historia canónica

En Event Sourcing el estado no se guarda únicamente como una fila mutable, sino como una secuencia de hechos de solo anexión [7, 8]. Si $e_1, \dots, e_n$ son los eventos de un run, su estado se reconstruye mediante una función reductora:

$$S_n = \text{fold}(S_0, [e_1, e_2, \dots, e_n]). \quad (2.1)$$

El journal registra hechos como la creación del run, el inicio de un intento, la adopción de un artefacto o la publicación final. Las pantallas y snapshots son proyecciones derivadas. Esta separación permite recuperar el sistema tras un reinicio, distinguir evidencia canónica de trazas diagnósticas y explicar con precisión cómo se alcanzó un resultado.

Capítulo 3

Estado del arte

ManyHands se ubica en la intersección entre agentes de programación, orquestación multi-agente, grafos de tareas e ingeniería de software basada en evidencia. Esta sección resume qué aporta cada línea y qué problema permanece abierto cuando se combinan sobre repositorios reales.

3.1. Agentes individuales de programación

SWE-bench consolidó una forma exigente de evaluar agentes: resolver incidencias históricas en repositorios reales y comprobar los parches mediante pruebas [11]. SWE-agent mostró además que la interfaz disponible para navegar y editar el repositorio influye fuertemente en el desempeño del modelo [19]. Herramientas como Codex CLI y Claude Code continúan esta línea mediante ciclos de inspección, edición y prueba.

Estas herramientas son adecuadas como *ejecutores*: reciben un objetivo y operan sobre un entorno. No resuelven necesariamente la gobernanza de varios intentos concurrentes, la propiedad de interfaces compartidas o la integración jerárquica. ManyHands las utiliza como componentes dentro de un plano de control, en lugar de competir con su capacidad generativa.

3.2. Sistemas multi-agente conversacionales

ChatDev y MetaGPT distribuyen roles como producto, arquitectura, desarrollo y pruebas entre agentes que intercambian mensajes y documentos [16, 9]. Su aporte principal es mostrar que la especialización de roles puede estructurar la generación de software.

La conversación, sin embargo, es una representación débil para coordinar cambios físicos. Una frase como “el módulo A provee usuarios” no fija una firma, una ruta ni una versión. Dos agentes pueden compartir la intención y aun así producir implementaciones incompatibles. ManyHands conserva la semántica en lenguaje natural durante la planificación, pero la compila a contratos y relaciones que el software puede verificar.

3.3. Grafos y paralelismo

LLMCompiler representa llamadas a herramientas mediante un DAG y ejecuta en paralelo aquellas cuyos datos de entrada ya están disponibles [12]. El principio es aplicable a agentes de código, pero el artefacto intercambiado ya no es sólo un valor: puede ser un conjunto de archivos, una interfaz pública o un commit.

Por ello, un grafo de software necesita distinguir tres preguntas: si una tarea consume el resultado de otra, si ambas pueden trabajar contra un contrato común, y si pretenden escribir el mismo recurso. ManyHands expresa estas preguntas mediante relaciones distintas, de modo que el scheduler no confunda dependencia funcional con conflicto de escritura.

3.4. Asignación de modelos y granularidad

FrugalGPT y RouteLLM estudian cómo dirigir una consulta hacia un modelo que equilibre calidad y costo [5, 14]. Esa decisión responde *qué modelo* debe atender una unidad. La política de granularidad de esta tesis responde una pregunta anterior: *qué cantidad de trabajo* debe constituir una unidad. Ambas estrategias son complementarias.

3.5. Posicionamiento de ManyHands

La tabla 3.1 resume el foco de cada enfoque. No pretende establecer una competencia de rendimiento, sino ubicar el aporte arquitectónico de la tesis.

Cuadro 3.1: Posicionamiento conceptual de ManyHands.

Enfoque	Fortaleza principal	Aspecto que aborda ManyHands
Agente individual	Resolución iterativa de una incidencia	Coordinación e integración de varias unidades
Multi-agente conversacional	Especialización y cooperación por roles	Contratos ejecutables y trazabilidad física
DAG de herramientas	Paralelismo por dependencias de datos	Artefactos Git, interfaces y conflictos de escritura
Enrutamiento de modelos	Selección costo-capacidad	Definición previa del tamaño de cada unidad

El aporte distintivo de ManyHands es combinar tres ideas que suelen aparecer separadas: descomposición explicable, coordinación mediante contratos y entrega respaldada por evidencia sobre el commit exacto. El resultado es un plano de control orientado a la ingeniería del proceso, no un nuevo modelo generativo.

Capítulo 4

Política de granularidad adaptativa

Este capítulo aborda el segundo eje de investigación de esta tesis: el diseño, la evolución y la formalización de la política de granularidad de descomposición. Se analiza críticamente el fracaso de las fórmulas tradicionales basadas en puntuaciones de utilidad continua, se presenta la **Política de Granularidad Categórica 4.0** fundada en propiedades estructurales observables, se detalla el algoritmo de colapso y remapeo de contratos, y se expone la metodología del banco de regresión offline que garantiza la reproducibilidad de las decisiones.

4.1. La tensión fundamental: cuándo y cómo dividir

Como se estableció en la introducción, descomponer un objetivo de software en unidades de trabajo para agentes autónomos enfrenta la paradoja de la granularidad. Una descomposición no es un fin en sí mismo, sino un mecanismo para hacer viable la ejecución de un agente dentro de sus límites cognitivos y computacionales.

Para formalizar esta decisión, es necesario responder dos preguntas estructurales:

1. **¿Cuándo debe mantenerse una unidad como una única hoja cohesiva?** Cuando la tarea puede ser resuelta de manera confiable por un solo agente dentro de sus presupuestos de contexto y tiempo, y dividirla no aporta paralelismo real ni aislamiento en la verificación.
2. **¿Cuándo debe aceptarse una partición en sub-tareas?** Únicamente cuando la división permita que sub-tareas independientes comiencen de forma concurrente, cuando aisle evidencias de prueba para que el fallo de una parte no anule el éxito de otra, o cuando la unidad supere la capacidad operativa de un intento individual.

4.2. Crítica a los modelos continuos de scoring heurístico

En etapas tempranas del diseño de **ManyHands** (versión 3 y anteriores), la decisión de granularidad se modelaba como una función de utilidad continua tradicional. Se definía un *índice de complejidad intrínseca* $C_{task}(u) \in [0, 10]$ para cada unidad u , calculado como una combinación lineal ponderada de cuatro dimensiones continuas:

$$C_{task}(u) = w_1 S_r(u) + w_2 I_i(u) + w_3 V_s(u) + w_4 T_m(u) \quad (4.1)$$

donde S_r representaba el radio de alcance de archivos, I_i el impacto de interfaz pública, V_s la superficie de validación y T_m la masa de contexto en tokens, con pesos normalizados $w_1 = 0,30$,

$w_2 = 0,25$, $w_3 = 0,25$ y $w_4 = 0,20$. Dicho índice se comparaba contra un umbral arbitrario $\theta = 3,5$: si $C_{task}(u) > \theta$ la unidad se dividía; en caso contrario, se preservaba como hoja.

4.2.1. El análisis empírico sobre 83 cortes históricos

Durante la evaluación sistemática de esta fórmula contra un banco de 83 cortes de descomposición registrados en ejecuciones históricas reales, se descubrieron tres fallas metodológicas insalvables:

1. **Adimensionalidad y falta de significado físico:** El número resultante de la ecuación 4.1 era un escalar adimensional sin unidades físicas. No expresaba cuántos tokens se ahorran, cuántos minutos de reloj se ganaban ni qué probabilidad de fallo se mitigaba. Un corte no podía justificarse formalmente porque el costo de coordinación no tenía la misma unidad que el beneficio de dividir.
2. **Insensibilidad del umbral:** Al recorrer los valores previstos por el diseño, amplios intervalos producían exactamente las mismas decisiones. Un parámetro que puede cambiar sin alterar el comportamiento no aporta una calibración defendible.
3. **Efecto difuso e injustificable:** En los 83 casos evaluados, todo el aparato matemático de cuatro dimensiones y pesos solo aportó 10 colapsos de unidades frente a la regla trivial «dividir siempre que el corte sea viable», y ningún factor o dimensión explicaba consistentemente esos colapsos.

Esta evidencia condujo a una conclusión teórica central: **la granularidad no es un problema de optimización continua sobre números reales, sino una decisión discreta y estructurada fundamentada en propiedades categóricas del corte y límites empíricos de capacidad operativa.**

4.3. La Política de Granularidad Categórica 4.0

La versión canónica de la política de granularidad (`granularity/4.0.0`), implementada en `@manyhands/decomposer`, elimina los parámetros libres, las ponderaciones y los umbrales numéricos continuos. Cada decisión de partición o colapso se sostiene en **tres razones categóricas independientes**, formulables en lenguaje natural y auditables por el operador:

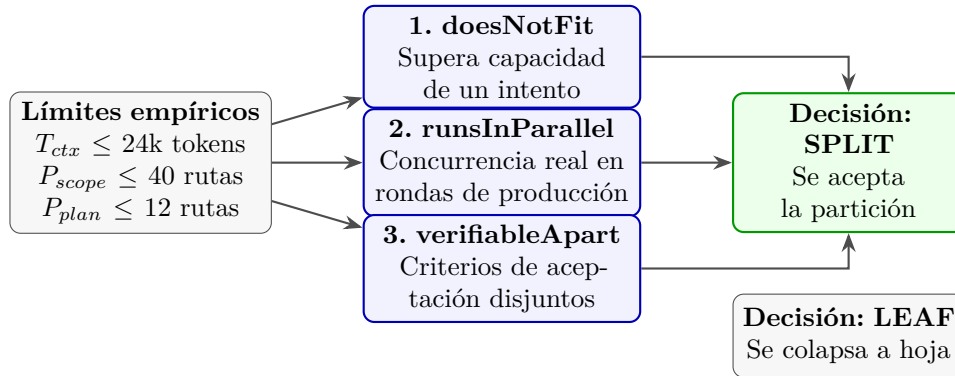


Figura 4.1: Estructura de decisión de la Política de Granularidad Categórica 4.0.

4.3.1. Límites empíricos de viabilidad de hoja (*Leaf Feasibility Bounds*)

Los únicos parámetros cuantitativos que permanecen en la política no son ponderaciones subjetivas de juicio, sino cotas empíricas calibradas que describen la capacidad operativa máxima que un único agente puede sostener de forma confiable antes de experimentar degradación atencional:

$$\text{EsViable}(u) \iff \begin{cases} \text{TokensMedidos}(u) \leq \text{maxLeafContextTokens} & (24\,000 \text{ tokens}) \\ \text{RutasEnAlcance}(u) \leq \text{maxLeafScopePaths} & (40 \text{ archivos}) \\ \text{RutasACrear}(u) \leq \text{maxLeafPlannedPaths} & (12 \text{ archivos}) \end{cases} \quad (4.2)$$

Lectura frente a creación de archivos: La separación explícita entre `RutasEnAlcance` (archivos existentes leídos o modificados) y `RutasACrear` (archivos nuevos que el agente debe generar) surgió de una lección histórica en la prueba piloto *Warehouse W2*. En dicho escenario, tras una refactorización inicial el repositorio era muy pequeño; por ende, la unidad raíz leía casi nada y cumplía holgadamente los límites de tokens y de alcance existente. Sin embargo, debía crear una aplicación completa desde cero con más de 15 archivos nuevos. El sistema evaluó la tarea como «hoja viable», colapsó la división en tres partes y despachó un único agente que consumió 30 minutos de presupuesto sin lograr entregar código funcional. La inclusión del límite empírico `maxLeafPlannedPaths = 12` previene que una tarea de creación masiva sea clasificada erróneamente como una hoja atómica.

4.3.2. Las tres razones categóricas de división

Una unidad compuesta solo se divide si se satisface al menos una de las siguientes tres propiedades categóricas:

1. No cabe en un intento (`doesNotFit`)

La unidad propuesta excede al menos uno de los tres límites físicos de viabilidad de la ecuación 4.2. En este caso, dividir no es una cuestión de preferencia o calidad: es la única forma de obtener una frontera ejecutable. Si la unidad no cabe y no existe ninguna propuesta viable de corte, el motor emite de inmediato un resultado determinista `semantic_replan`.

2. Ejecución concurrente real (`runsInParallel`)

El corte compra tiempo de reloj (*wall-clock*) permitiendo que al menos dos unidades hijas inicien simultáneamente. El cálculo de concurrencia se realiza de forma estricta sobre el grafo de dependencias de artefactos.

Sean $C = \{c_1, c_2, \dots, c_n\}$ los nodos hijos de la unidad compuesta y E_{art} las dependencias dirigidas inducidas por artefactos materializados entre ellos. Sea $\text{RondasProduccion}(C, E_{art})$ el número mínimo de etapas secuenciales requeridas para completar todas las unidades según su orden topológico. Decimos que el corte ofrece paralelismo real si y solo si:

$$\text{RondasProduccion}(C, E_{art}) < |C| \quad \wedge \quad \text{Aciclico}(C, E_{art}) \quad (4.3)$$

Si n unidades requieren exactamente n rondas secuenciales (una cadena estricta $c_1 \rightarrow c_2 \rightarrow \dots \rightarrow c_n$), el scheduler deberá ejecutarlas una tras otra. Una cadena puramente secuencial **no reduce el tiempo de reloj** y puede ser más lenta que un único intento debido al costo de preparar contextos y worktrees. Por tanto, recibe `runsInParallel = false`.

3. Verificación aislada e independiente (**verifiableApart**)

Cada unidad hija posee al menos un criterio de aceptación exclusivo que ningún nodo hermano comparte:

$$\forall c_i \in C, \quad \exists \text{crit} \in \text{Criterios}(c_i) \quad \text{tal que} \quad \forall c_j \in C \setminus \{c_i\}, \text{crit} \notin \text{Criterios}(c_j) \quad (4.4)$$

Si un hijo no posee ningún criterio propio, su éxito o fracaso no puede observarse independientemente de sus hermanos: separar dicha unidad no aporta nueva evidencia. Por el contrario, cuando cada hijo es dueño de una prueba disjunta, el fallo de una sub-tarea no invalida el trabajo ya verificado de las demás. Una partición en capas puede justificarse así aunque no reduzca el tiempo: compra **aislamiento de fallas y contención de regresiones**.

4.3.3. Regla de decisión determinista

La regla de decisión de la política 4.0 se formaliza de la siguiente manera:

$$R(u) = \text{doesNotFit} \vee \text{runsInParallel} \vee \text{verifiableApart},$$

$$\text{Decisión}(u) = \begin{cases} \text{split}, & \text{si } \text{CorteViable}(u) \wedge R(u), \\ \text{leaf}, & \text{si } \text{EsViable}(u) \wedge \neg R(u), \\ \text{semantic_replan}, & \text{en otro caso.} \end{cases} \quad (4.5)$$

Cada decisión emite automáticamente una explicación en lenguaje natural generada determinísticamente (**describeDecision**), indicando las razones exactas que sostuvieron el corte o el colapso (por ejemplo: «*Split because two children can start at the same time; every child owns a criterion no sibling owns*»).

4.4. Algoritmo de colapso y remapeo de planes

Cuando la política determina que un corte no se justifica ($\text{Decisión}(u) = \text{leaf}$), la función pura **applyGranularitySelection** aplica una poda estructural sobre el árbol del plan semántico (**SemanticPlan**):

La intuición del algoritmo es sencilla: al eliminar una frontera de coordinación también desaparecen sus hijos, pero no sus responsabilidades. Por eso la hoja resultante hereda todos los criterios y rutas que antes pertenecían a sus descendientes. Del mismo modo, una costura interna deja de tener sentido cuando productor y consumidor terminan dentro de la misma hoja. Este mecanismo preserva el principio de que **un criterio de aceptación nunca se pierde durante un colapso**.

4.5. Frontera entre semántica y control determinista

Una premisa de diseño esencial en ManyHands es la estricta separación de incumbencias:

El modelo de lenguaje propone la semántica del corte (qué módulos existen y qué responsabilidades asumen); la política determinista decide si la propuesta se acepta o se colapsa, basándose en capacidad, concurrencia y aislamiento.

Si la política detecta que una unidad es demasiado compleja pero el planificador no propuso sub-unidades, el sistema no sintetiza particiones artificiales dividiendo rutas al azar (estrategia

Algoritmo 4.1: Colapso determinista de unidades y remapeo de costuras

1. **Entrada:** Plan semántico candidato $\mathcal{P}$, Evaluaciones de granularidad $\mathcal{A}$.
2. Para cada unidad compuesta $u \in \mathcal{P}$ evaluada como **leaf**:
 - a) Transformar el rol de u a hoja atómica (**kind**: "leaf").
 - b) Fusionar y deduplicar en u todas las obligaciones de sus descendientes:

$$\text{RutasPlanificadas}(u) \leftarrow \bigcup_{v \in \text{Descendientes}(u)} \text{RutasPlanificadas}(v)$$

$$\text{Criterios}(u) \leftarrow \bigcup_{v \in \text{Descendientes}(u)} \text{Criterios}(v)$$

- c) Eliminar los nodos hijos de la jerarquía.
 3. Para cada costura o contrato de interfaz $s \in \mathcal{P}.\text{seams}$:
 - a) Remapear el productor y los consumidores de s a su ancestro sobreviviente más cercano.
 - b) Si el productor y los consumidores colapsaron en la misma unidad ($u_{\text{prod}} == u_{\text{cons}}$), **eliminar la costura** s (la interfaz se convirtió en una llamada interna dentro del mismo módulo).
 4. **Retorno:** Plan semántico podado y validado contra el esquema canónico.
-

que demostró inducir fallas de alcance en el pasado). En su lugar, el sistema solicita formalmente una replanificación semántica (**semantic_replan**) al motor de planificación o eleva una decisión estructurada al operador humano.

4.6. Metodología del banco de regresión offline

Para hacer auditable y reproducible la política sin incurrir en costos continuos de inferencia, se implementó un **banco de regresión offline**.

El banco almacena un corpus versionado de más de 80 casos históricos, con sus descomposiciones candidatas y estados de repositorio. Ante cualquier cambio en la política, una suite determinista reevalúa todos los casos y compara las decisiones con una línea base congelada. Toda divergencia falla en integración continua y exige revisar explícitamente el cambio antes de aprobarlo.

Capítulo 5

Arquitectura del sistema

Este capítulo presenta primero el sistema completo y luego profundiza en las decisiones que constituyen su aporte principal. La separación es deliberada: un lector puede seguir un *run* de punta a punta sin conocer el código y, al mismo tiempo, comprender por qué la granularidad, los contratos y el grafo tipado permiten coordinar trabajo concurrente sin perder control.

5.1. Vista integral: del objetivo a la entrega

En **ManyHands**, la unidad de interacción y valor es el **run**: la transformación auditable de un requerimiento en lenguaje natural y un commit base exacto en un resultado integrado, verificado y entregado. El sistema se organiza en capas con responsabilidades explícitas; la tabla 5.1 nombra todos sus subsistemas para que el resto del capítulo pueda concentrarse en sus relaciones, no en un inventario de archivos.

Cuadro 5.1: Mapa funcional completo de los 17 subsistemas de ManyHands.

Capa	Subsistemas	Responsabilidad conjunta
Interacción	apps/web	Command Center y Run Cockpit; valida la frontera web y proyecta eventos sin poseer la ejecución.
Host privilegiado	apps/daemon, windows-job-runner, windows-ipc-acl	Custodia el estado, el IPC autenticado, las credenciales y los árboles de procesos del sistema operativo.
Dominio durable	run-coordinator, run-engine, run-store, trace-store	Define eventos y transiciones puras, serializa comandos, persiste hechos y separa la telemetría diagnóstica.
Conocimiento y plan	shared, repository-index, decomposer, contracts, task-graph	Modela el repositorio, decide la descomposición y compila contratos en una revisión ejecutable del grafo.
Disponibilidad	scheduler, conflict-risk, orchestrator-graph	Calcula qué nodos están listos, prioriza sin alterar la seguridad y conduce las olas de ejecución e integración.
Ejecución y prueba	execution-core	Materializa entradas exactas, aísla intentos, construye artefactos Git, valida candidatos y publica la entrega.

El recorrido completo se resume en la tabla 5.2. Cada fila produce una identidad o evidencia que la siguiente debe consumir; por eso el flujo es también una cadena de custodia.

Cuadro 5.2: Recorrido end-to-end de un run.

	Etapa	Transformación observable
1	Ingreso y objetivo	La UI envía un comando; el núcleo registra un GoalContract ligado al repositorio y commit base.
2	Grounding	repository-index construye un RepositoryModel y vistas exactas, declarando también lo desconocido.
3	Plan semántico	decomposer consulta esa vista y propone responsabilidades, contratos, artefactos, costuras y recursos.
4	Verificación y compilación	verifyPlan rechaza patologías; el compilador produce una única GraphRevision inmutable.
5	Frontera ejecutable	scheduler evalúa precondiciones duras y orchestrator-graph selecciona nodos compatibles para la siguiente ola.
6	Intento de hoja	execution-core materializa sólo las entradas declaradas, ejecuta al agente en aislamiento y crea un candidato desde el diff observado.
7	Adopción e integración	La evidencia decide la adopción; composites y raíz integran artefactos frescos mediante nuevos intentos sobre candidatos exactos.
8	Validación y entrega	La Matriz de Evidencias vincula obligaciones con el árbol final; el publicador aplica una actualización Git atómica y emite un recibo.
9	Persistencia y supervisión	El daemon conserva eventos y efectos; la UI reconstruye proyecciones y actividad, incluso tras una recarga o reinicio.

A diferencia de un chat efímero, el run está gobernado por eventos y una máquina de estados determinista. Puede atravesar planificación, decisiones acotadas, ejecución, reparación, integración, validación y entrega. Una decisión bloquea sólo los nodos afectados; una integración puede comenzar cuando sus insumos están frescos, sin esperar una barrera global. El estado **completed** sólo existe después del recibo de entrega correspondiente al candidato verificado.

5.2. Modelo de grafo híbrido y relaciones canónicas tipadas

Un grafo en ManyHands no es una colección homogénea de nodos conectados por aristas arbitrarias. Se estructura como un **grafo híbrido jerárquico**:

- **Nodo Raíz (*Goal Root*)**: Representa el objetivo general del usuario y la envoltura final de integración.
- **Nodos Compuestos (*Composite Nodes*)**: Representan fronteras de integración arquitectónica. No realizan implementación ordinaria de una hoja, pero su intento de integración puede modificar recursos compartidos que el contrato asignó explícitamente al padre.
- **Nodos Hoja (*Leaf Nodes*)**: Representan unidades atómicas de implementación asignadas a un agente individual dentro de un único worktree aislado.

5.2.1. Tipado canónico de relaciones

Para eliminar la ambigüedad de las dependencias genéricas, el grafo tipifica estrictamente cuatro clases de relaciones formales (tabla 5.3):

Esta distinción tipada resuelve un problema fundamental en la ingeniería de software multi-agente: un acuerdo de interfaz (**SeamBinding**) no impone una espera temporal y un reclamo de recurso expresa autoridad, no una dependencia implícita que el scheduler pueda inventar.

Un ejemplo permite ver la diferencia. Si una tarea implementa un servicio y otra su interfaz web, ambas pueden comenzar en paralelo cuando comparten un contrato de API: eso es una costura. Si la interfaz necesita un archivo generado por el servicio para ejecutar

Cuadro 5.3: Catálogo de relaciones canónicas del grafo de ManyHands.

Relación	Semántica formal	Efecto operativo en el Scheduler	Orden temporal
Jerarquía (parentId)	Pertenencia jerárquica y propiedad de integración.	El padre espera a que todos sus hijos finalicen para ejecutar la integración.	Ascendente (<i>bottom-up</i>)
ArtifactRequirement	Un nodo consumidor requiere un archivo/artefacto producido por otro.	Bloquea el despacho del consumidor hasta que el artefacto exista físicamente.	Estricto ($A \rightarrow B$)
SeamBinding	Productor y consumidores comparten un contrato de interfaz congelado.	Habilita ejecución concurrente: ambos nodos trabajan contra el contrato formal.	Paralelo ($A \parallel B$)
ResourceClaim	Un nodo declara qué versión observa o qué recurso puede modificar.	Valida autoridad única; una transición ordenada exige consumir el artefacto anterior.	Propiedad y versión

sus pruebas, existe además un requisito de artefacto y debe esperar. Si dos tareas pretenden modificar `package.json`, el plan debe asignar esa edición compartida al padre o modelar una transformación versionada en la que la segunda consume el artefacto de la primera. Dos propietarios paralelos son inválidos aunque se ejecuten uno después del otro. Una flecha genérica no podría expresar correctamente estas situaciones.

5.3. Grounding del repositorio: RepositoryModel y consultas presupuestadas

Para evitar que los modelos de lenguaje alucinen la estructura de directorios o los nombres de las clases exportadas, la planificación se fundamenta en un modelo formal del repositorio provisto por `@manyhands/repository-index`:

1. **Snapshot inmutable (RepositorySnapshot):** Representa una fotografía exacta del repositorio anclada al SHA del commit base de Git. El descubrimiento de rutas se realiza mediante `ripgrep` nativo, indexando archivos rastreados e ignorando rutas declaradas en `.gitignore`.
2. **Vista de repositorio (RepositoryView):** Estructura que clasifica los límites de paquetes, módulos exportados, catálogos de recursos y capacidades de validación detectadas (por ejemplo, scripts de test en `package.json`, configuraciones de Vitest o pytest).
3. **Consultas presupuestadas (RepositoryQuery):** Durante la fase de planificación, el modelo LLM no recibe el código fuente completo como contexto estático. En su lugar, el planificador emite consultas acotadas al índice (`searchGoalTerms`, `inspectBoundary`, `validationCapabilities`). Cada consulta descuenta bytes leídos y llamadas contra un presupuesto estricto (`RepositoryQueryBudget`), de modo que el consumo permanezca dentro de límites explícitos.

5.4. Motor de planificación progresiva presupuestada (PlanningEngine)

La generación de planes se implementa en `PlanningEngine`. El componente pertenece al paquete `@manyhands/decomposer` y, en vez de un único prompt generativo (*one-shot*), el motor alterna propuestas, consultas y revisiones. Un `PlanningBudget` limita llamadas al modelo, consultas y bytes leídos del repositorio, revisiones, reparaciones y expansiones. El presupuesto convierte la exploración abierta en un algoritmo con costo y terminación explícitos.

5.4.1. Prevención de bucles causales (*Causal Loop Prevention*)

Uno de los modos de fallo más comunes en la planificación asistida por LLMs es el atrapamiento en ciclos infinitos de reparación: ante un error de validación, el modelo propone una corrección que reintroduce un error previo, oscilando indefinidamente sin converger.

Para detectar este comportamiento, `PlanningEngine` calcula en cada iteración un digest criptográfico del estado causal:

$$\text{causalDigest} = \text{SHA256}(\text{proposalDigest} \parallel \text{requestDigest} \parallel \text{queryReceipts} \parallel \text{findings}) \quad (5.1)$$

El motor mantiene un conjunto inmutable de estados visitados $\mathcal{S}_{\text{seen}}$. Si una nueva propuesta produce un $\text{causalDigest} \in \mathcal{S}_{\text{seen}}$, aborta la rama con un hallazgo tipado `no_progress`. Así evita repetir un estado sin avance; los presupuestos completan el límite de terminación del proceso.

5.4.2. Taxonomía discriminada de resultados de planificación

El resultado retornado por `PlanningEngine.plan()` es una unión discriminada estricta que modela con precisión matemática el estado de la propuesta:

- **ready:** El plan cumple el 100 % de los invariantes y es directamente compilable a un grafo ejecutable.
- **needs_input:** El plan identificó ambigüedades de negocio que requieren clarificación humana; genera un objeto inmutable `PlanningContinuation` con el digest de continuación.
- **ambiguous:** Existen múltiples alternativas arquitectónicas viables pero mutuamente incompatibles.
- **unsupported:** El repositorio carece de las herramientas o capacidades mínimas exigidas por el requerimiento.
- **rejected:** El plan violó invariantes estáticos no reparables o agotó su presupuesto asignado.

5.5. Sistema canónico de contratos: acotando los modos de falla

En un sistema multi-agente, la delegación no puede depender únicamente de descripciones informales. El lenguaje natural conserva la intención, pero no ofrece por sí solo una frontera ejecutable. Los contratos actúan como la traducción entre ambos mundos: preservan el significado

propuesto durante la planificación y lo convierten en condiciones que el plano de control puede validar.

Para neutralizar estas amenazas, **ManyHands** formaliza cada dimensión de la ejecución mediante un catálogo inmutable de contratos validados mediante esquemas Zod en `@manyhands/contracts` (figura 5.1):

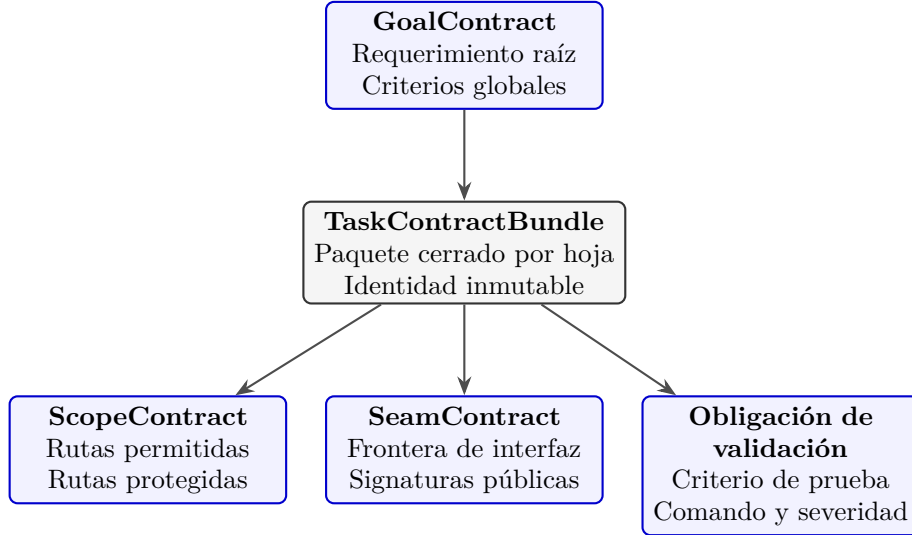


Figura 5.1: Jerarquía de contratos inmutables en ManyHands.

Cada contrato responde a una decisión explícita de diseño arquitectónico:

1. **GoalContract (Preservación del requerimiento raíz):** Fija la intención global del usuario y los criterios de aceptación innegociables. Impide que la descomposición sufra de *deriva teleológica*, asegurando que ninguna sub-tarea descarte objetivos originales.
2. **TaskContractBundle (Hermeticidad del agente):** Constituye el paquete cerrado e inmutable entregado al worker. Un agente no tiene visibilidad del grafo completo ni de las conversaciones de otros agentes: su universo operacional está estrictamente delimitado por su bundle.
3. **ScopeContract (Contención de dispersión de archivos):** Declara las listas blancas de rutas de lectura y escritura (`writePaths`). Combinado con la inspección del diff, permite rechazar ediciones ajenas a la incumbencia del agente.
4. **SeamContract (Congelamiento de interfaces públicas):** Formaliza tipos exportados, firmas de métodos y endpoints HTTP acordados *a priori*. Al desacoplar la interfaz de su implementación, permite que productor y consumidores trabajen en paralelo (`SeamBinding`); la compatibilidad efectiva se comprueba luego en integración.
5. **ArtifactContract (Determinismo de materialización):** Especifica los archivos físicos que el nodo debe generar y sus modos de persistencia, para que la materialización pueda comprobar las rutas esperadas.
6. **CanonicalValidationObligation (Gobernanza probatoria):** Vincula cada criterio de aceptación con un comando de validación exacto y una política de severidad. Impide que un agente declare una tarea como completada sin evidencia física ejecutable.

5.6. Verificador estático de invariantes (`verifyPlan`): prevención de patologías de grafo

Antes de convertir un plan semántico en una revisión ejecutable del grafo (`GraphRevision`), la función pura `verifyPlan` revisa sus invariantes. La estrategia es equivalente a un compilador: el modelo propone una estructura expresiva; el verificador rechaza estados que no podrían ejecutarse de manera segura. Ningún plan generado por un LLM atraviesa esta barrera sólo por parecer razonable en lenguaje natural.

Las ocho categorías se resumen en la tabla 5.4. La autoridad de recursos merece una precisión: dos escritores superpuestos sin orden son inválidos. Una transformación secuencial sólo es válida si el segundo consume explícitamente el artefacto versionado del primero; el scheduler no “repara” una doble propiedad limitándose a ejecutar en serie.

Cuadro 5.4: Invariantes estáticos verificados antes de ejecutar.

Verificación	Estado inválido que evita
Jerarquía y aciclicidad	Raíces múltiples, huérfanos o ciclos que impiden integrar.
Criterios	Pérdida, debilitamiento o invención de objetivos durante la descomposición.
Roles y granularidad	Hojas que coordinan o composites sin una obligación real de integración.
Artefactos	Dependencias circulares y consumidores sin un productor materializable.
Costuras	Interfaces sin productor, consumidor o prueba de compatibilidad.
Rutas	Escapes del repositorio, rutas absolutas y acceso a zonas protegidas.
Recursos	Autoridad de escritura superpuesta o versiones de entrada ambiguas.
Evidencia	Criterios obligatorios sin estrategia y autoridad de prueba aceptadas.

Una vez superadas las ocho categorías de invariantes, la función `compilePlan` transforma determinísticamente el plan semántico en una `GraphRevision`. De este modo, la ejecución concurrente sólo recibe planes que atravesaron la barrera estática definida por el sistema.

Capítulo 6

Implementación

Este capítulo recorre la implementación en el mismo orden que un run. Explica cómo los módulos del mapa de la tabla 5.1 convierten comandos en hechos durables, hechos en trabajo físico y resultados físicos en evidencia y entrega. El énfasis no está en enumerar clases, sino en las fronteras que impiden perder estado, confundir un intento con otro o validar un commit distinto del que finalmente recibe el usuario.

6.1. Organización del código y fronteras de paquetes

ManyHands está implementado en TypeScript bajo el entorno de ejecución Node.js (versión 20+ ESM), estructurado como un monorepo gestionado con **pnpm workspaces**. Para asegurar la mantenibilidad y prevenir dependencias circulares, se impone una regla estricta de direccionalidad unidireccional de dependencias: **apps** → **packages especializados** → **packages compartidos**.

La capa **shared** contiene primitivas sin dependencias, serialización canónica y tipos epistémicos comunes. Sobre ella se apoyan contratos y grafo; los paquetes de dominio no dependen de Next.js, React ni de una SDK de modelos. Así, cambiar la interfaz o el proveedor de agentes no cambia el significado de un evento, un contrato o una evidencia. La direccionalidad se complementa con una segunda regla: existe una sola representación canónica en cada frontera (**SemanticPlan**, **GraphRevision**, **RunEvent** y manifiestos Git), evitando modelos paralelos que puedan divergir.

6.2. Núcleo durable: eventos, actor y efectos

@manyhands/run-coordinator implementa el dominio puro. Sus comandos poseen identidad e indican la revisión esperada; sus eventos son hechos inmutables; y **reduceRun** repliega cualquier secuencia válida en la misma **RunProjection**. Consultar no modifica el estado y repetir un comando idéntico devuelve el recibo previo, mientras que reutilizar su identidad con otro contenido se rechaza.

@manyhands/run-engine aporta el **RunActor**. Cada run tiene un buzón secuencial y un único escritor autorizado, aunque distintas operaciones físicas ocurran en paralelo. El actor no ejecuta directamente Git, modelos o procesos. Primero registra una intención durable **effect.requested**; un adaptador realiza el efecto y produce un recibo; finalmente el actor incorpora el resultado como evento. Esta separación evita el intervalo peligroso en el que una operación externa ocurrió pero el sistema no dejó constancia.

@manyhands/run-store conserva el journal autoritativo en JSONL, con *fsync*, reemplazos

atómicos, tokens de cercado y un *outbox* de efectos. Las instantáneas, índices y vistas de UI son proyecciones reconstruibles, no una segunda verdad. Al reiniciar, el daemon reconcilia efectos no terminales con los recibos y el estado físico antes de repetirlos. No presupone una falsa semántica *exactly once*: o demuestra que repetir es idempotente, o adopta el resultado observado, o solicita una decisión.

`@manyhands/trace-store` guarda actividad, tiempos, consumo y diagnóstico en un canal separado, con checksums y redacción de secretos. Una traza ayuda a explicar qué ocurrió, pero nunca puede promover un intento ni cambiar el ciclo de vida: esa autoridad pertenece exclusivamente a los eventos del run.

6.3. Disponibilidad, priorización y conducción de olas

`@manyhands/scheduler` evalúa continuamente la disponibilidad de cada nodo. Comprueba contratos aprobados, artefactos frescos, decisiones resueltas, autoridad de recursos, capacidad del ejecutor, presupuesto y concesiones físicas. El resultado no es una fase global, sino una frontera de nodos *hard-ready* que cambia después de cada evento.

`@manyhands/conflict-risk` estima interferencias por rutas, símbolos y dependencias para priorizar dentro de esa frontera. Su salida es consultiva: puede reducir concurrencia o cambiar el orden, pero no inventar dependencias ni autorizar escrituras. `@manyhands/orchestrator-graph` conduce el bucle: selecciona una ola compatible, solicita intentos, incorpora artefactos adoptados y vuelve a calcular la frontera. De ese modo hojas e integraciones intermedias pueden alternarse sin esperar a que termine todo el grafo.

6.4. Intentos, artefactos y ejecución aislada

Cada ejecución, reparación o integración crea un intento inmutable. Su `InputFingerprint` resume nodo, contratos, árbol base, vista del repositorio, manifiestos consumidos, ejecutor y capacidad de sandbox. Si cambia una entrada, se crea otro intento y la evidencia anterior queda obsoleta; un intento terminal nunca se reabre.

`ExecutionBaseBuilder` materializa un árbol exacto desde el commit base y únicamente los manifiestos declarados. El agente edita, pero no decide la identidad del resultado: el orquestador observa el diff, verifica alcance y autoridad, y recién entonces crea el commit candidato. `ArtifactBuilder` extrae objetos Git y los describe en manifiestos inmutables que conservan archivos binarios, permisos, enlaces y eliminaciones. El estado de adopción o validación se registra aparte, para no mutar la identidad del artefacto.

La ejecución de código generado plantea riesgos de corrupción del sistema de archivos y desbordes de recursos. Por eso se aplican capas independientes de aislamiento y custodia:

6.4.1. Pool de Git worktrees y reciclado determinista

Cada intento asignado a un agente se despacha en un worktree independiente dentro de un pool administrado por el orquestador. Antes de iniciar la ejecución, el worktree se restablece al commit base requerido mediante una secuencia atómica de comandos Git:

Cada slot se materializa desde una base Git identificada y se entrega a un solo intento bajo una concesión exclusiva. Cuando el agente finaliza, el orquestador inspecciona el árbol mediante `git diff` y `git status --porcelain=v1`. La salida textual del modelo nunca se usa para decidir qué archivos cambiaron: **la fuente física de verdad es el diff del worktree.**

Antes de reutilizar un slot, el gestor comprueba que no existan operaciones Git incompletas ni propietarios todavía activos.

6.4.2. Verificación estricta de alcance en tiempo de ejecución (*Scope Guard*)

Para evitar que un agente modifique archivos ajenos a su sub-tarea, el subsistema de ejecución aplica un validador de alcance en dos fases con política de *denegación prioritaria* (*deny-first*):

1. **Normalización POSIX y prevención de escapes:** Cada ruta tocada se normaliza resolviendo enlaces simbólicos (*symlinks*) y secuencias de ascenso de directorios (*..*). Cualquier intento de escapar de la raíz del worktree genera un rechazo inmediato.
2. **Verificación contra el contrato de alcance:** Todo archivo modificado, creado o eliminado en el diff debe pertenecer al conjunto de rutas permitidas (`writePaths`) del `ScopeContract`. Si el agente toca un solo archivo no autorizado (por ejemplo, `package.json` o archivos de configuración global cuando solo tenía permiso sobre `src/domain/`), el intento se aborta de inmediato clasificándose con el estado terminal `scope_violation`, y ningún commit es adoptado por el orquestador.

6.4.3. Supervisión de procesos, comunicación IPC y saneamiento de entorno

- **Supervisión del árbol de procesos del SO:** La CLI del agente se inicia como un proceso supervisado. En Windows, un *Job Object* nativo mantiene la custodia del árbol completo; si vence un plazo o el usuario cancela, el cierre del trabajo termina también los descendientes. El efecto sólo se considera físicamente final después de comprobar la identidad y el resultado del proceso supervisado.
- **Sandbox y credenciales acotadas:** El sandbox de ejecución limita la escritura al worktree. La credencial necesaria para invocar al modelo se copia a un hogar efímero identificado por run e intento y se revoca después del resultado físico terminal. Así, el worker no necesita heredar el entorno completo del daemon ni conservar autenticación entre intentos.
- **Concesiones con testigo (*Witness Leases*):** Toda mutación sobre un repositorio o estado del run exige una concesión durable con vigencia demostrable [3]. Si un worker tarda demasiado o el daemon se reinicia, cualquier resultado tardío se descarta.

Un worktree separa checkouts; no constituye por sí solo un sandbox. La restricción de archivos, la custodia de procesos, la política de red y el alcance de credenciales son capacidades distintas y se verifican por separado. En Windows, `windows-job-runner` aporta la custodia nativa de procesos y `windows-ipc-acl` protege los *Named Pipes* mediante DACLs acotadas al usuario y al sistema, incluyendo defensas frente a redirecciones del sistema de archivos.

6.5. Integración ascendente y reparación semántica de conflictos

La integración es un intento de primer orden modelado en el grafo. Los nodos compuestos no son agrupadores visuales: construyen un candidato combinado desde los manifiestos frescos de sus hijos y, cuando el contrato lo prevé, realizan ediciones sólo sobre recursos compartidos propiedad del padre:

1. **Composición en árbol limpio:** El nodo compuesto materializa una base exacta e integra los cambios declarados de sus hijos siguiendo el orden topológico de artefactos, verificando preimágenes y versiones.
2. **Verificación de costuras e interfaces:** Una vez fusionados los cambios, el compuesto ejecuta las obligaciones de validación de nivel integración (`layer: integration`) asociadas a sus `SeamContracts`. Que dos ramas apliquen sin conflictos textuales de Git no garantiza corrección semántica: la integración solo se considera exitosa si las pruebas de integración conjuntas pasan.
3. **Reparación semántica de conflictos (*Semantic Conflict Repair*):** Si la composición o una prueba falla, el orquestador crea un nuevo intento con los diagnósticos y el contexto del padre. Una corrección necesaria sobre recursos de un hijo vuelve a ese hijo o exige una enmienda; la integración no recibe permisos universales.

6.6. Recuperación causal y decisiones durante el run

Los fallos no comparten un contador universal de reintentos. El dominio los clasifica por causa: entrada inválida, entorno o proveedor transitorio, violación de alcance, proceso perdido, incompatibilidad de integración o evidencia insuficiente, entre otras. Reintentar es válido sólo si cambia una condición relevante, por ejemplo el contexto, la capacidad disponible o el diagnóstico entregado al agente. Así se evita repetir indefinidamente la misma ejecución.

La recuperación conserva el trabajo ya adoptado cuyos fingerprints continúan vigentes. Un error de una hoja no obliga a reiniciar el run: puede originar una reparación local; una modificación de contrato genera una revisión o `Amendment` inmutable y vuelve obsoletos únicamente los descendientes afectados. Cuando la resolución requiere criterio humano, una `Decision` declara los nodos alcanzados y permite que las ramas independientes sigan avanzando.

6.7. Matriz de Evidencias (Evidence Matrix)

La validez de un resultado de software en ManyHands se sintetiza en la **Matriz de Evidencias** (`EvidenceMatrix`). Cada celda de la matriz vincula formalmente:

$$\text{CeldaEvidencia} = (\text{CriterioId}, \text{ObligacionValidacionId}, \text{CommitSha}, \text{Resultado}, \text{TrazaComando}) \quad (6.1)$$

La matriz aplica las siguientes reglas de auditoría:

- **Prohibición de pruebas debilitadas (*Test Integrity Guard*):** El verificador de evidencias audita los diffs de pruebas generados por los agentes. Si detecta la eliminación de aserciones existentes, la adición de modificadores para omitir pruebas (como `it.skip`, `describe.only` o `xit`) o la modificación de configuraciones del runner de pruebas para ignorar fallos, el resultado de validación es clasificado como `fraudulent_evidence` y rechazado de inmediato.
- **Controles negativos cuando son aplicables:** Una prueba nueva puede ejecutarse sobre el commit base para comprobar que detecta la ausencia del cambio. Si pasa también sobre el código previo, no sirve como evidencia discriminante para esa obligación.

- **Política contra pruebas inestables (*Flaky Test Policy*):** Las obligaciones de validación críticas declaran `flakyPolicy`: "forbid". Si una prueba exhibe intermitencia no determinista durante repeticiones consecutivas sobre el mismo commit, la matriz clasifica la celda como inestable y bloquea la promoción del artefacto.

Un candidato sólo alcanza el estado `verified` cuando todas las obligaciones requeridas están satisfechas sobre ese candidato. Las observaciones `advisory` o `inconclusas` pueden conservarse para diagnóstico, pero no sustituyen una obligación bloqueante.

6.8. Publicación y Entrega Transaccional (TransactionalDeliveryPublisher)

La fase final de entrega es gestionada por la clase `TransactionalDeliveryPublisher` en `@manyhands/execution-core`. La publicación garantiza atomicidad y trazabilidad mediante el protocolo de tres fases ilustrado en la figura 6.1:

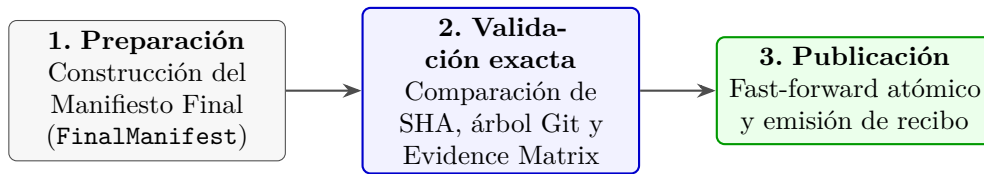


Figura 6.1: Protocolo de publicación transaccional de entrega.

1. **Manifiesto de artefacto final (FinalArtifactManifest):** Documento inmutable que contiene el SHA del commit candidato, el SHA del árbol físico de Git (`treeSha`), el ID de la Matriz de Evidencias verificada y la rama de destino.
2. **Validación de concurrencia y precondiciones:** El publicador inspecciona el repositorio de destino y comprueba que la rama destino no haya avanzado concurrentemente (`targetHead == targetHeadBefore`) y que el directorio de trabajo esté limpio.
3. **Publicación y Recibo de Entrega (TransactionalDeliveryReceipt):** El publicador ejecuta un avance rápido atómico (`git merge -ff-only <finalSha>`) y emite un recibo inmutable persistido en el journal de eventos, conteniendo las identidades criptográficas de antes y después del cambio.

6.9. Persistencia append-only basada en eventos y Daemon local

El estado se persiste mediante el *Event Store* de `@manyhands/run-store` en formato JSON Lines, con semántica de sólo anexión. Cada lote se publica de manera segura:

$$\text{archivo_temp} \xrightarrow{\text{flush a disco}} \text{fsync()} \xrightarrow{\text{reemplazo atómico}} \text{archivo_definitivo} \quad (6.2)$$

El **daemon local** es el anfitrión privilegiado y único dueño de actores, journals, temporizadores, credenciales intermediadas y workers. Una concesión de instalación impide que dos daemons posean el mismo directorio de estado. En el arranque, reconcilia journals y efectos pendientes antes de aceptar comandos.

`apps/web` no se conecta desde JavaScript al canal privilegiado. Su servidor Next.js funciona como BFF: valida `Host`, `Origin`, token de sesión y tipo de contenido, y recién entonces traduce comandos y consultas a IPC local autenticado con HMAC, nonce y protección contra repetición. En Windows, el transporte usa un *Named Pipe* con ACL protegida. Si el navegador se cierra o recarga, el daemon sigue ejecutando y la vista se reconstruye desde la proyección durable.

6.10. Interfaz web de supervisión y accesibilidad

La interfaz gráfica está construida sobre Next.js y React Flow como un **Command Center**. Los comandos cambian el mundo; las consultas y el stream ordenado de eventos sólo lo observan. SSE conserva el número de secuencia y, tras una desconexión o un hueco, vuelve a consultar la proyección antes de continuar. Por eso una recarga no crea otra verdad ni depende de memoria local:

- **Estabilidad visual del lienzo:** Se implementa el invariante estricto de que **el lienzo del grafo nunca se recentra, reencuadra ni altera su nivel de zoom de forma automática** ante la llegada de eventos de ejecución o cambios de estado de los nodos. El desarrollador mantiene el control absoluto de su encuadre de trabajo.
- **Cola global de decisiones no bloqueante:** Las decisiones humanas pendientes se presentan en una cola accesible. El trabajo en ramas independientes del grafo continúa ejecutándose concurrentemente: una decisión pendiente detiene únicamente aquellos nodos cuya disponibilidad depende directamente del resultado de dicha decisión.
- **Accesibilidad y movimiento reducido:** La interfaz fue diseñada siguiendo WCAG 2.2 AA [17]: utiliza nombres accesibles, foco visible, contraste suficiente y respeta `prefers-reduced-motion`. Estas decisiones permiten supervisar el sistema sin depender exclusivamente del color o de animaciones.
- **Estados honestos y observabilidad:** La UI distingue candidato, verificado, obsoleto, fallido y entregado, y vincula cada nodo con su actividad y diagnóstico. Costos y trazas ayudan a operar el run, pero no se presentan como prueba de corrección.

Capítulo 7

Evaluación experimental: Viaje en Familia

Este capítulo presenta una demostración de extremo a extremo de ManyHands. El objetivo no fue medir qué modelo escribe más código, sino observar si el plano de control podía convertir un requerimiento amplio en un grafo útil, ejecutar trabajo concurrente, conservar resultados válidos durante las reparaciones, integrar las partes y publicar un commit reproducible.

7.1. Diseño del caso

El experimento se definió antes de la ejecución. El repositorio inicial sólo contenía un servidor estático, un `package.json` sin dependencias y una prueba basal. El requerimiento solicitó construir *Viaje en Familia*, una aplicación local en HTML, CSS y JavaScript ESM con persistencia en `localStorage`. El dominio se organizó alrededor de tres áreas:

- **Ruta e itinerario:** paradas, agenda y búsqueda;
- **Organización:** presupuesto y equipaje;
- **Recuerdos:** notas y favoritos.

La raíz debía integrar esas áreas en un dashboard y validar un recorrido completo. Esta forma del requerimiento permitía observar los tres niveles buscados: objetivo, fronteras de integración y hojas cohesivas.

7.2. Configuración y fuente de evidencia

El run se inició desde la interfaz web sobre un repositorio Git limpio. La tabla 7.1 resume la configuración persistida en el evento `run.created`.

La autoridad del análisis es el journal canónico de 254 eventos. Las trazas de los modelos se utilizaron para diagnosticar intentos, pero no para decidir el estado del run. Los commits, manifiestos, matrices y recibos se contrastaron con el repositorio físico.

7.3. Topología producida

La planificación generó un grafo de 13 nodos: una raíz, tres compuestos intermedios y nueve hojas. La figura 7.1 resume la jerarquía de propiedad.

Cuadro 7.1: Configuración del run Viaje en Familia.

Dimensión	Valor
Planificación	Claude Code, modelo Sonnet
Ejecución y reparación	Codex CLI, GPT-5.4 Mini, esfuerzo medio
Autonomía	Autónoma
Paralelismo máximo	Seis intentos
Política de alcance	Estricta
Base Git	c9ae12ac14f44e23c1e5243fd055a1c438904af4
Run	run:1572bf9195031800...f961487eb3ae0

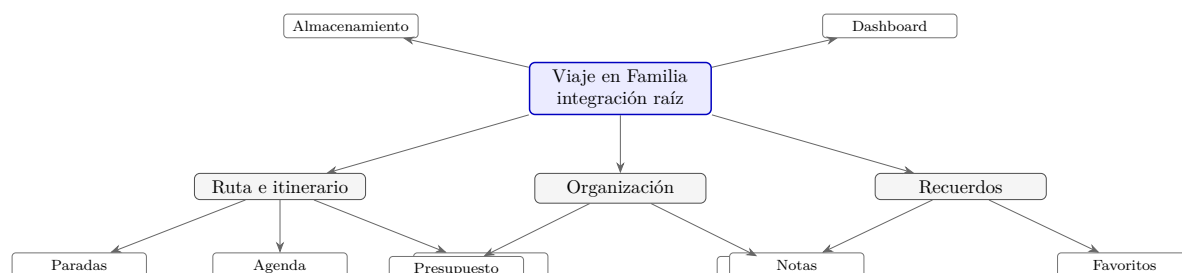


Figura 7.1: Jerarquía simplificada del grafo generado.

La topología no se limitó a agrupar tarjetas. Cada compuesto poseía una obligación de integración y consumía los artefactos de sus hijos. Las costuras de almacenamiento, agenda, presupuesto, equipaje, notas y favoritos fijaron las fronteras compartidas; los reclamos de recursos evitaron escritores superpuestos.

7.4. Ejecución, paralelismo y recuperación

El scheduler seleccionó 22 olas a lo largo del run. La primera construyó el almacenamiento común. Luego ejecutó en una misma frontera presupuesto, equipaje y paradas; más adelante agenda y notas compartieron otra ola, y favoritos y búsqueda otra. Esta evidencia muestra paralelismo estructural: se ejecutaron unidades independientes, no copias de una misma tarea.

El experimento también ejerció la recuperación. Algunos intentos de paradas y dashboard fallaron por validaciones o por límites de tiempo. Cada fallo produjo una decisión tipada y un nuevo intento; los artefactos que ya estaban verificados permanecieron adoptados. El sistema no volvió a planificar el objetivo ni repitió ramas completas. Esta propiedad fue especialmente visible en el dashboard: las reparaciones se concentraron en ese nodo mientras organización, itinerario y recuerdos conservaron sus resultados.

El primer intento de integración raíz detectó una aserción incorrecta en el recorrido conjunto y quedó registrado como fallido. Un segundo intento corrigió la composición y volvió a ejecutar la validación sobre un candidato nuevo. Este comportamiento ilustra el valor práctico del modelo durable: fallar no equivale a perder el run; equivale a producir nueva evidencia y reabrir únicamente la frontera afectada.

7.5. Integraciones y candidato final

Las integraciones intermedias finalizaron antes de la raíz, como muestra la tabla 7.2.

La raíz produjo un candidato cuyo identificador abreviado fue 62a0d357; su árbol Git fue 58dd2f76. La Matriz de Evidencias `matrix-da779f2d70dfd21c` vinculó el criterio raíz con

Cuadro 7.2: Resultados de integración del run.

Frontera	Candidato	Matriz
Organización	7ea56ef2...	verified
Ruta e itinerario	df924593...	verified
Recuerdos	fc967e23...	verified
Integración raíz	62a0d357...	verified

el test end-to-end ejecutado sobre ese commit y con su control negativo. Los identificadores completos permanecen en el journal para auditoría.

Tras verificar el candidato final, el sistema inició y publicó la entrega. El recibo confirmó un avance de la rama `main` desde la base al candidato, sin modificar su árbol. Por lo tanto, candidato, entrega y repositorio final compartieron la misma identidad.

7.6. Reproducción y alcance del resultado

La suite del target ejecutó 32 pruebas: almacenamiento, paradas, agenda, búsqueda, presupuesto, equipaje, notas, favoritos, integraciones de las tres áreas y recorrido end-to-end. Las 32 aprobaron sobre el commit entregado. Luego se creó un clon limpio mediante `git clone --no-local`; el clon resolvió el mismo SHA, permaneció sin archivos sin rastrear y volvió a aprobar 32 de 32 pruebas.

La observación browser confirmó que el servidor se inicia con `npm start`, que el dashboard presenta estados vacíos y los resúmenes de las tres áreas, y que la pantalla se adapta a la estructura prevista. La evidencia confirmatoria de esta tesis se limita a esa presentación y a las pruebas automatizadas del recorrido de lógica. Una evaluación de usabilidad interactiva completa, con participantes y medición sistemática en múltiples viewports, queda fuera de las conclusiones experimentales.

En conjunto, el caso demuestra el recorrido que la tesis se propuso estudiar: un objetivo amplio fue convertido en contratos y un grafo de tres niveles; varias ramas avanzaron en paralelo; las fallas se recuperaron sin descartar trabajo válido; las integraciones se verificaron de abajo hacia arriba; y un commit exacto fue publicado y reproducido desde un clon limpio.

Capítulo 8

Discusión

8.1. Qué demuestra el sistema

El aporte principal de ManyHands no es generar más líneas de código ni reemplazar la capacidad de los modelos utilizados como ejecutores. Su aporte es una cadena de custodia para el trabajo generado:

$$\text{Objetivo} \rightarrow \text{Plan} \rightarrow \text{Contratos} \rightarrow \text{Intentos} \rightarrow \text{Evidencia} \rightarrow \text{Integración} \rightarrow \text{Entrega.} \quad (8.1)$$

El run Viaje en Familia muestra que esta cadena puede operar de extremo a extremo. La planificación produjo una jerarquía coherente; el scheduler encontró fronteras concurrentes; los compuestos integraron a sus hijos; la recuperación reabrió nodos fallidos sin descartar artefactos adoptados; y la entrega quedó vinculada a un commit y un árbol Git reproducibles.

El resultado no implica que toda ejecución autónoma de software vaya a completar cualquier objetivo. Demuestra una proposición más acotada y útil: es posible construir un plano de control que mantenga trazabilidad y permita recuperar fallas durante una colaboración multi-agente real.

8.2. Modelos para semántica, software para autoridad

Una decisión transversal del diseño fue separar dos responsabilidades:

- el modelo interpreta el objetivo, propone límites semánticos y escribe o repara código;
- el plano determinista valida esquemas, calcula disponibilidad, limita recursos, inspecciona Git y decide si existe evidencia suficiente.

La separación evita pedir al mismo componente probabilístico que produzca el resultado y, al mismo tiempo, sea la autoridad final sobre su corrección. El modelo conserva libertad dentro de un intento; los contratos y verificadores limitan las consecuencias que ese intento puede tener sobre el run.

8.3. Por qué la granularidad es categórica

La experiencia con el puntaje continuo mostró que una fórmula puede parecer precisa sin ser explicable. Sumar cantidad de archivos, tokens, interfaces y pruebas produce un número,

pero no una razón física para dividir. La política categórica cambia la pregunta: en lugar de “¿qué puntaje obtuvo esta tarea?”, pregunta “¿qué beneficio concreto compra la separación?”.

Las tres respuestas posibles —capacidad, paralelismo o aislamiento de validación— pueden explicarse a un operador y observarse después de la ejecución. Además, el algoritmo de colapso reduce el costo de una propuesta demasiado fragmentada sin perder los criterios que debían verificarse.

8.4. Costo de control y recuperación

Los worktrees, journals, validaciones y recibos agregan tiempo respecto de una invocación única. Este costo es deliberado. En un parche trivial puede ser innecesario; en un cambio con varias fronteras, permite atribuir resultados, aislar fallas y continuar después de una interrupción.

El experimento también muestra que la recuperación tiene costo propio. Un nodo puede requerir varios intentos antes de converger, y una evidencia mal especificada puede bloquear código correcto. La ventaja no es eliminar el fallo, sino volverlo local, durable y diagnosticable. La principal oportunidad de mejora consiste en reducir intentos repetidos mediante mejores diagnósticos y checkpoints de candidatos parciales.

8.5. Alcance de la evidencia

La evidencia técnica es fuerte en aquellas propiedades vinculadas a Git y al journal: topología, orden de eventos, procesos, candidatos, matrices, entrega y reproducción. Las 32 pruebas del target cubren la lógica de las áreas y su composición. La revisión visual confirma la presentación del dashboard y sus estados básicos.

La evaluación no constituye un estudio de usabilidad con usuarios ni una comparación estadística contra otros orquestadores. Tampoco demuestra escalabilidad a repositorios de millones de líneas o a todos los lenguajes. Estas distinciones protegen la interpretación del caso: el experimento aporta evidencia de viabilidad arquitectónica, no una ley universal sobre rendimiento de modelos.

8.6. Amenazas a la validez

- **Variabilidad del modelo.** Una nueva ejecución puede proponer código o particiones diferentes. El freeze de configuración y el journal reducen ambigüedad, pero no eliminan la estocasticidad.
- **Caso único.** Viaje en Familia fue elegido por su riqueza estructural. Otros dominios pueden producir grafos y costos distintos.
- **Pruebas como aproximación.** Una suite verde no mide por sí sola mantenibilidad o experiencia de uso. Por eso se separan las conclusiones de ingeniería de las futuras evaluaciones cualitativas.
- **Entorno local.** La supervisión y el sandbox fueron ejercitados en Windows. La portabilidad conceptual no sustituye una calificación física equivalente en otros sistemas operativos.

Estas amenazas no invalidan el recorrido observado, pero delimitan qué puede inferirse a partir de él.

Capítulo 9

Conclusiones

Esta tesis partió de una dificultad concreta: un objetivo de software puede ser demasiado amplio para un único agente, pero dividirlo sin criterio crea más problemas de los que resuelve. ManyHands aborda esa tensión mediante una arquitectura en la que la semántica generativa y el control determinista cumplen funciones distintas.

9.1. Respuesta a las preguntas de investigación

Respecto de la primera pregunta, el trabajo muestra que es viable coordinar agentes mediante un grafo jerárquico, contratos versionados, ejecución aislada e integración basada en evidencia. La viabilidad no se apoya sólo en el diseño: el run experimental completó planificación, ejecución, recuperación, integraciones intermedias, integración raíz y entrega sobre un commit exacto.

Respecto de la segunda pregunta, se formuló una política de granularidad determinista y explicable. La decisión ya no depende de un puntaje difícil de interpretar, sino de tres razones observables: que la tarea no quepa, que la división habilite paralelismo o que permita verificar partes por separado. El algoritmo de colapso conserva obligaciones y elimina fronteras sin utilidad.

9.2. Aportes principales

Los aportes del trabajo pueden resumirse en seis elementos:

1. un **grafo híbrido** que separa jerarquía, flujo de artefactos, contratos de interfaz y conflictos de recursos;
2. una **política de granularidad categórica** acompañada por un algoritmo de colapso auditable;
3. un **catálogo de contratos** que preserva objetivo, alcance, artefactos y criterios durante la descomposición;
4. un subsistema de **ejecución aislada y supervisada** sobre Git worktrees, sandbox y custodia del árbol de procesos;
5. una **integración ascendente** que trata a los compuestos como intentos de primer orden y vincula criterios con evidencia;
6. una **historia durable del run** y una entrega transaccional que relacionan el resultado publicado con su candidato físico.

9.3. Lecciones de ingeniería

Tres ideas atravesaron el desarrollo. Primero, una declaración del modelo nunca debe sustituir la inspección del artefacto. Segundo, los contratos reducen la ambigüedad sólo si están conectados con mecanismos físicos de ejecución y validación. Tercero, un sistema autónomo útil no es aquel que nunca falla, sino el que puede localizar una falla, conservar el trabajo válido y continuar sin perder trazabilidad.

El experimento final reunió estas ideas en un único recorrido. El resultado fue un grafo de tres niveles, con trabajo concurrente, integraciones por área, reparaciones localizadas, candidato verificado y entrega reproducible. Más que una demostración visual, el valor del caso reside en que cada etapa puede explicarse mediante eventos, commits y recibos.

9.4. Trabajo futuro

Las extensiones prioritarias son:

1. incorporar oráculos de producto externos como obligaciones canónicas previas a la entrega;
2. mejorar los checkpoints de intentos para reutilizar candidatos parciales durante una reparación;
3. calificar el sandbox y la supervisión en más sistemas operativos;
4. estudiar la política sobre repositorios y lenguajes diversos;
5. evaluar usabilidad y comprensión del command center con usuarios;
6. combinar granularidad con enrutamiento dinámico de modelos según costo y capacidad.

ManyHands no elimina la incertidumbre de los modelos de lenguaje. La rodea con una estructura de ingeniería que vuelve sus acciones observables, recuperables y verificables. Ese es el resultado central de esta tesis y la base para futuras plataformas de desarrollo autónomo responsables.

Bibliografía

- [1] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [2] Frederick P. Brooks. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, anniversary edition, 1995.
- [3] Mike Burrows. The chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 335–350, 2006.
- [4] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2 edition, 2014.
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [7] Martin Fowler. Event sourcing. *martinfowler.com*, 2005.
- [8] Pat Helland. Life beyond distributed transactions: An apostate’s opinion. In *Conference on Innovative Data Systems Research (CIDR)*, pages 132–141, 2007.
- [9] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [12] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *International Conference on Machine Learning (ICML)*, 2024.
- [13] Bertrand Meyer. Applying “design by contract”. *Computer*, 25(10):40–51, 1992.

- [14] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.
- [15] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- [16] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15174–15186, 2024.
- [17] W3C. Web content accessibility guidelines (WCAG) 2.2. W3c recommendation, World Wide Web Consortium, 2023.
- [18] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [19] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.
- [20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.